Chapter 8. Railway-oriented processing

This chapter covers

- Handling failure within Unix programs, Java exceptions, and error logging
- Designing for failure inside and across stream processing applications
- Composing failures inside work steps with the Scalaz `Validation`
- Failing fast across work step boundaries with Scala's `map` and `flatMap`

So far, we have focused on what you might call the *happy path* within our unified log. On the happy path, events successfully validate against their schemas, inputs are never accidentally null, and Java exceptions are so rare that we don't mind them crashing our application.

The problem with focusing exclusively on the happy path is that failures *do* happen. More than this: if you implement a unified log across your department or company, failures will happen *extremely frequently*, because of the sheer volume of events flowing through, and the complexity of your event stream processing. Linus's law states, "Given enough eyeballs, all bugs are shallow."[1] Adapting this, a law of unified log processing might be as follows:

> *1*
>
> *Linus's law is further explained at Wikipedia, https://en.wikipedia.org/wiki/Linus%27s_Law.*
>
> *Given enough events, all bugs are inevitable.*

If we can expect and design for inevitable failure inside our stream processing applications, we can build a much more robust unified log, one that hopefully won't page us regularly at 2 a.m. because it crashed Yet Another `NullPointerException` (YANPE). This chapter, then, is all about designing for failure, using an overarching approach that we will call *railway-oriented processing*. We have been using this approach at Snowplow throughout our event pipeline since the start of 2013, so we know that it can enable the processing of billions of events daily with a minimum of disruption.

For reasons that should become clear as we delve deeper into this topic, this chapter will be the first one where we work predominantly in Scala. Scala is a strongly typed, hybrid object-oriented and functional language that runs on the Java virtual machine. Scala has great support for railway-oriented processing—capabilities that even Java 8 lacks.

So, all aboard the railway-oriented programming express, and let's get started.

8.1. Leaving the happy path

> *Two roads diverged in a wood, and I—*
>
> *I took the one less traveled by,*
>
> *And that has made all the difference.*
>
> *Robert Frost, "The Road Not Taken" (1916)*

Before diving into railway-oriented processing, let's look at how failure is handled in two distinct environments that many readers will be familiar with: Unix programs and Java. We'll then follow this up with general thoughts on error logging as it is practiced today.

8.1.1. Failure and Unix programs

Unix is designed around the idea that failures will happen. Any process that runs in a Unix shell will return an exit code when it finishes. The convention is to return zero for success, and an integer value higher than zero in the case of failure.

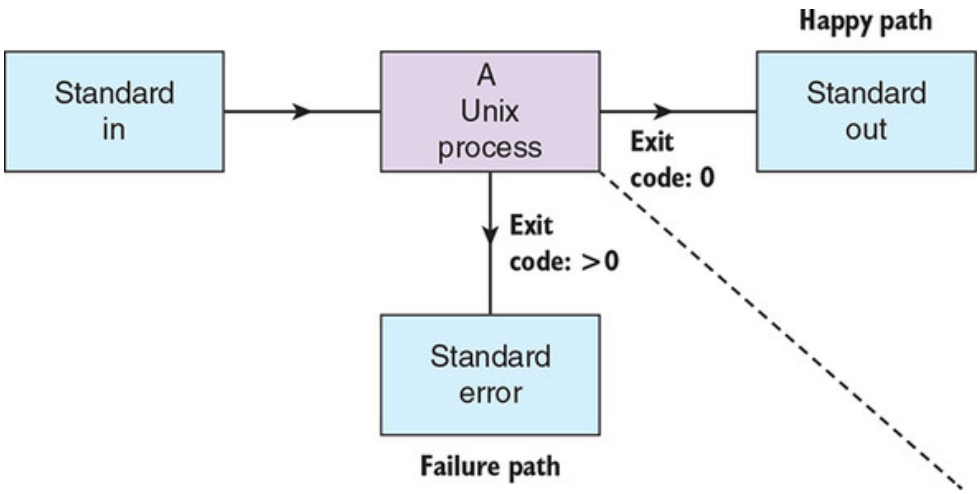
This exit, or return, code isn't the only communication channel available to a Unix program; each program also has access to three standard streams (aka I/O file descriptors): `stdin`, `stdout`, and `stderr`. Table 8.1 provides the properties of these three streams.

Table 8.1. The three standard streams supported by Unix programs

Short name	Long name	File descrip-tor	Description
stdin	Standard in	0	The input stream of data going into a Unix program
stdout	Standard out	1	The output stream where a Unix program writes data related to successful operation
stderr	Standard error	2	The output stream where a Unix program writes data related to failed operation

Putting the exit codes and the three standard streams together, we can represent a Unix program's happy and failure paths, as shown in figure 8.1.

Figure 8.1. A Unix program reads from standard in and can respond with exit codes and output streams along a happy path or a failure path.



It's only correct to note, however, that things are not always as clear-cut as figure 8.1 implies:

- A Unix process that ends up failing with a nonzero exit code may well also write output to `stdin` before it fails.
- Likewise, a chatty Unix process may write warnings or diagnostic output to `stderr` before ultimately returning with a zero code indicating success.

How *composable* is failure handling in Unix programs? By *composable*, we mean that can we combine multiple successes and failures, and the combined result will still make sense. Let's see—if you have a Unix terminal handy, type in the following:

```
$ false | false | true; echo $?
0
```

If you are not familiar with `false` and `true`, these are super-simple Unix programs that return exit codes of `1` (failure) and `0` (success), respectively. In this example, we are combining two `false` programs with a `true` program, using the vertical bar character `|` (also known as a *pipe*) to make a *Unix pipeline*.

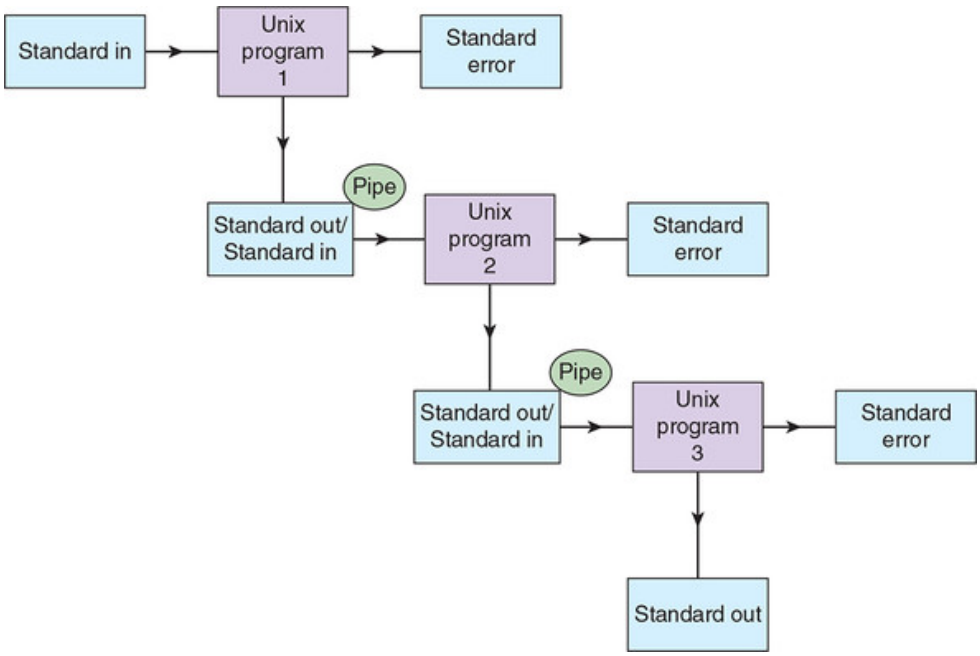
As you can see, the first two failures do not cause the pipeline to fail, and the ultimate return code of the pipeline is the return code of the last program run.

In some Unix shells (ksh, zsh, bash), we can do a little better than this by using the built-in `pipefail` option:

```
$ alias info='>&2 echo'
$ set -o pipefail; info s1 | false | info s3 | true; echo $?
s3
s1
1
```

The `pipefail` option sets the exit code to the exit code of the last program to exit nonzero, or returns `0` if all exited successfully. This is an improvement, but the `s3` output shows that the pipeline is still not short-circuiting, or failing fast, after an individual component within it fails. This is because a Unix pipeline is chaining input and output streams together, *not* exit codes, as demonstrated in figure 8.2.

Figure 8.2. Three Unix programs form a single Unix pipeline by piping the `stdout` of the first program into the `stdin` of the next program. Many shells support an option to pipe both `stdout` and `stderr` into `stdin`, but in both cases the program's exit codes are ignored.



If we do want to fail fast, we have to put our commands in a shell script that uses the `set –e` option, which will terminate the script as soon as any command within the script returns a nonzero error code. If you run the code in the following listing, you will see the following output; notice that the second `echo` in the shell script is never reached:

```
$ ./fail-fast.bash; echo $?
s1
1
```

Listing 8.1. fail-fast.bash

```
#!/bin/bash
set -e

echo "s1"
false
echo "s3"        1
```

- ***1*** **This line is never reached.**

In sum: Unix programs and the Unix pipeline have powerful yet easy-to-understand features for dealing with failure. But they are not as composable as we would like.

8.1.2. Failure and Java

Let's take a look now at how Java deals with failure. Java depends heavily on exceptions for handling failures, making a distinction between two types of exceptions:

- ***Unchecked exceptions***— `RuntimeException`, `Error`, and their subclasses. Unchecked exceptions represent bugs in your code that a caller cannot be expected to recover from: the dread `NullPointerException` is an unchecked exception.
- ***Checked exceptions***— `Exception` and its subclasses, except `RuntimeException` (which, strangely, is a subclass of `Exception`). In Java, every method must declare any uncaught checked exceptions that can be thrown within its scope, passing the responsibility on to the caller to handle them as they decide.

Let's look at the following `HelloCalculator` app, where we employ both unchecked and checked exceptions. The following listing contains the HelloCalculator.java code, annotated to show the use of both exception types.

Listing 8.2. HelloCalculator.java

```
package hellocalculator;

import java.util.Arrays;

public class HelloCalculator {

  public static void main(String[] args) {
    if (args.length < 2) {
      String err = "too few inputs (" + args.length + ")";
      throw new IllegalArgumentException(err);                    1
```

```
      } else {
        try {
          Integer sum = sum(args);
          System.out.println("SUM: " + sum);
        } catch (NumberFormatException nfe) {                    2
          String err = "not all inputs parseable to Integers";
          throw new IllegalArgumentException(err);               1
        }
      }
    }

    static Integer sum(String[] args) throws NumberFormatException {   3
      return Arrays.asList(args)
        .stream()
        .mapToInt(str -> Integer.parseInt(str))                  4
        .sum();
    }
  }
```

- ***1*** **IllegalArgumentException is an unchecked RuntimeException, which will cause our program to terminate.**
- ***2*** **NumberFormatException is a checked Exception, so we have to catch it here.**
- ***3*** **This method could throw a NumberFormatException, but we don't catch it, so we have to declare it as part of our method's API.**
- ***4*** **Integer.parseInt's own API declares that it could throw a NumberFormatException.**

Java's built-in failure handling is broadly designed around two failure scenarios:

- We have an unrecoverable bug and we want to terminate the program.
- We have a potentially recoverable issue and we want to try to recover from it (and if we can't recover from it, we terminate the program).

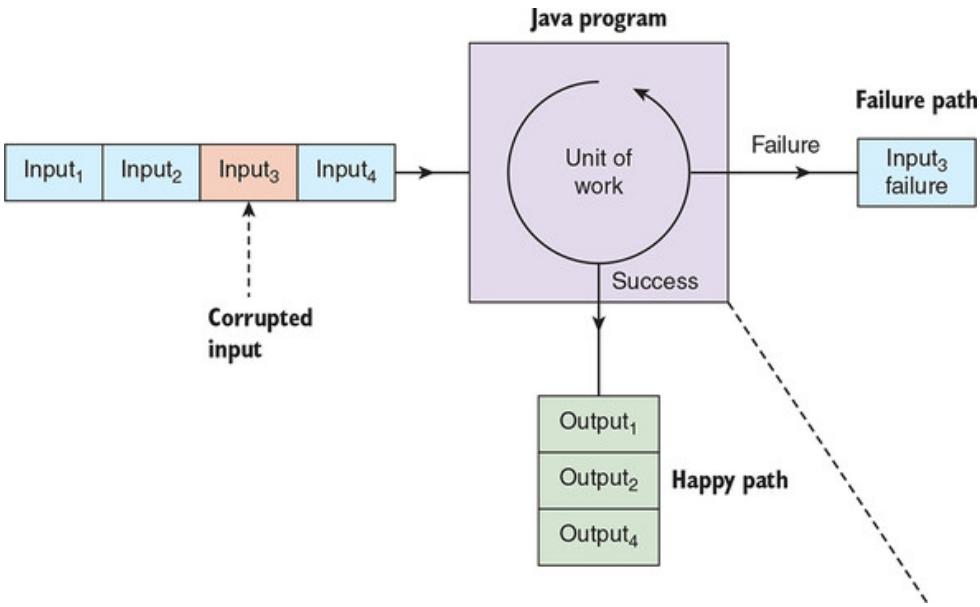
But what if we cannot recover from a failure but *don't* want to terminate our overall program? This might sound counterintuitive: how can our program keep going with an *unrecoverable failure*? To be sure, many programs cannot—but some can, especially if they consist of processing many much smaller *units of work*, for example:

- A web server, responding to thousands of HTTP requests a minute. Each request-response exchange is a unit of work.
- A stream processing job in our unified log, tasked with enriching many millions of individual events. The enrichment of each incoming event is a unit of work.

- A web-scraping bot, parsing thousands of web pages to look for product price information. The parsing of each web page is a unit of work.

In each of these scenarios, the programmer may prefer to route the unrecoverable unit of work to a failure path rather than terminate the whole program, as shown in figure 8.3.

Figure 8.3. Our Java program is processing four items of input, one of which is somehow corrupted. Our Java program performs the unit of work on each of the four inputs: three are successfully output along the happy path, but the corrupted input throws an error and ends up on the failure path.



As with many other languages, Java does not have any built-in tools for routing failing units of work to a failure path. In situations like these, a Java programmer will often fall back to simply logging the failure as an error and skipping to the next unit of work. To demonstrate this, let's imagine an updated version of the original HelloCalculator code: instead of summing all arguments together, our new version will increment (add 1 to) any numeric arguments, and log an error message for any arguments that are not numeric. This is illustrated in the following listing.

Listing 8.3. HelloIncrementor.java

```
package hellocalculator;

import java.util.Arrays;
import java.util.logging.Logger;                                    1

public class HelloIncrementor {

  private final static Logger LOGGER =
    Logger.getLogger(HelloIncrementor.class.getName());

  public static void main(String[] args) {
```

```
    Arrays.asList(args)
      .stream()
      .forEach((arg) -> {                                          2
        try {
          Integer i = incr(arg);
          System.out.println("INCREMENTED TO: " + i);              3
        } catch (NumberFormatException nfe) {
          String err = "input not parseable to Integer: " + arg;
          LOGGER.severe(err);                                      4
        }
      });
  }

  static Integer incr(String s) throws NumberFormatException {
    return Integer.parseInt(s) + 1;
  }
}
```

- ***1*** **We use the built-in java.util.Logging for our error logging.**
- ***2*** **Loop through all command-line arguments.**
- ***3*** **If we can increment, print out the incremented value.**
- ***4*** **If we cannot increment, log the error at the highest log level.**

Copy the code from listing 8.3 and save it into a file, like so:

```
hellocalculator/HelloIncrementor.java
```

Next, we will compile our code and run it, supplying three valid and one invalid (non-numeric) argument:

```
$ javac hellocalculator/HelloIncrementor.java
$ java hellocalculator.HelloIncrementor 23 52 a 1
INCREMENTED TO: 24
INCREMENTED TO: 53
Nov 13, 2018 5:42:30 PM hellocalculator.HelloIncrementor lambda$main$0
GRAVE: input not parseable to Integer: a
INCREMENTED TO: 2
```

See how our failure is buried in the middle of our successes? We have to look carefully at the program's output to discern the failure output.

Worse, we have just outsourced this program's failure path to our logging framework to handle. We can say that the failure path is now *out-of-band*: the failures are no longer present in the source code or influencing the program's *control flow*. Incidentally, many experienced programmers criticize the concept of exceptions for a similar reason: they create a secondary control flow in a program, one that exists outside the standard imperative flow, and thus is difficult to reason about.[2]

[2]

Joel Spolsky's essay on why exceptions are not always a good thing: https://www.joelonsoftware.com/2003/10/13/13/

Whatever the criticisms of exceptions, error logging is surely worse: it is not just outside the program's main control flow, but also outside the program itself. The challenges of dealing with out-of-band error logging have fueled a whole software industry, which we will briefly look at next.

8.1.3. Failure and the log-industrial complex

A Java program, a Node.js program, and a Ruby program all walk into a bar. Each logs an error on the way in. The joke is on the barman, because they are all speaking different languages:

```
ERROR  2018-11-13 06:50:14,125 [Log_main]  "com.acme.Log": Error from Jav
Error from node.js
E, [851000 #0] ERROR -- : Error from Ruby
```

Amazingly, no common format exists for program logging: different languages and frameworks all have their own logging levels (`error`, `warning`, and so forth) and log message formats. Combine this with the fact that log messages are written using human language, and you are left with logs that can often be analyzed using only plain-text search.

Various logistical issues also are associated with outsourcing your failure path to a logging framework:

- What if we run out of space on our server to store logs?
- In a world of transient virtual servers and ephemeral stateless containers, how do we ensure that we collect our logs before the server itself is terminated?
- How do we collect logs from client devices that we don't control?

Collectively dealing with these issues around logging have spawned what we might call, only half-jokingly, the *log-industrial complex*, consisting of the following:

- ***Logging frameworks and facades***— Java alone has Log4j, SLF4J, Logback, java.util .Logging, tinylog, and others.
- ***Log collection agents and frameworks***— These include Apache Flume, Logstash, Filebeat, Facebook's now-shuttered Scribe project, and Fluentd.
- ***Log storage and analytics tools***— These include Splunk, Elasticsearch plus Kibana, and Sawmill.

- ***Error collection services***— These include Sentry, Airbrake, and Rollbar. These services are often focused on collecting errors from client devices.

Even with all of this tooling, in a unified log context we still have another unsolved problem: how can we reprocess a failed unit of work (for example, an event) if and when the underlying issue that caused the failure is fixed?

In a unified log world, there has to be a better way of working with our failure path. We will explore this next.

8.2. Failure and the unified log

> *Reports that say that something hasn't happened are always interesting to me, because as we know, there are known knowns; there are things we know we know. We also know there are known unknowns; that is to say, we know there are some things we do not know. But there are also unknown unknowns—the ones we don't know we don't know.*
>
> *US Secretary of Defense Donald H. Rumsfeld (February 12, 2002)*

In this section, we will set out a simple pattern for unified log processing that accounts for the failure path as well as the happy path. For our treatment of the failure path, we will borrow from the better ideas of section 8.1 and throw in some new ideas of our own.

8.2.1. A design for failure

How should we be handling failure in our unified log processing? First, let's propose some rules governing *program termination*. We should terminate our job only if one of the following occurs:

- We encounter an unrecoverable error during the initialization phase of our job.
- We encounter a novel error while processing a unit of work inside our job.

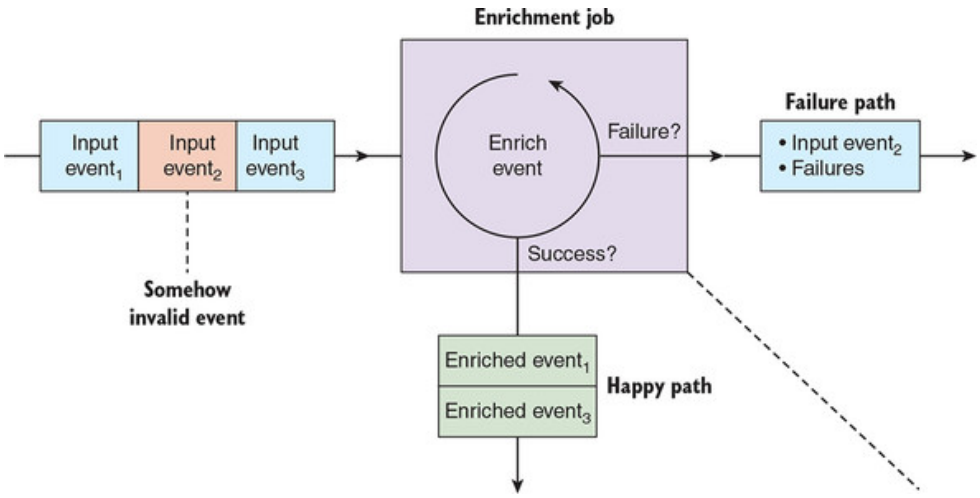
A *novel error* means one that we haven't seen before: a Rumsfeld-esque unknown unknown. Although it might be tempting to keep processing in this case to minimize disruption, terminating the job forces us to evaluate any novel error as soon as we encounter it. We can then determine how this new error should be handled in the future—for example, can it be recovered from without failing the unit of work?

Next, how do we handle an unrecoverable but not-unexpected error within a unit of work? Here there is no way around it—we have to move this unit of work into our failure path, but making sure to follow a few important rules:

- Our failure path must not be out-of-band. We don't need to rely on third-party logging tools; we are implementing a unified log, so let's use it!
- Entries in our failure path must contain the reason or reasons for the failure in a well-structured form that both humans and machines can read.
- Entries in our failure path must contain the original input data (for example, the processed event), so that the unit of work can potentially be replayed if and when the underlying issue can be fixed.

Let's make this a little more concrete with the example of a simple stream-processing job that is enriching individual events, as shown in figure 8.4.

Figure 8.4. Our enrichment job processes events from the input event stream, writes enriched events to our happy path event stream, and writes input events that failed enrichment, plus the reasons why they failed enrichment, to our failure path event stream.



Does the flow in figure 8.4 look familiar? It shares a lot in common with Unix's concept of three standard streams: our stream processing app will *read* from one event stream, and *write* to two event streams, one for the happy path and the other for individual failures. At the same time, we have improved some of the failure-handling techniques of section 8.1:

- ***We have done away with exit values.*** The success or failure of a given unit of work is reflected in whether output was written to the happy event stream or the failure event stream.
- ***We have removed any ambiguity in the outputs.*** A unit of work results in output *either* to the happy stream or to the failure stream, nev-

er both, nor neither. Three input events mean a total of three output events.
- ***We are using the same in-band tools to work with both our successes and our failures.*** Failures will end up as well-structured entries in one event stream; successes will end up as well-structured entries in the other event stream.

This is a great start, but what do we mean when we say that our failures will end up as "well-structured entries" in an event stream? Let's explore this next.

8.2.2. Modeling failures as events

How could we describe our stream processing job's failure to enrich an event read from an input stream? Perhaps something like this:

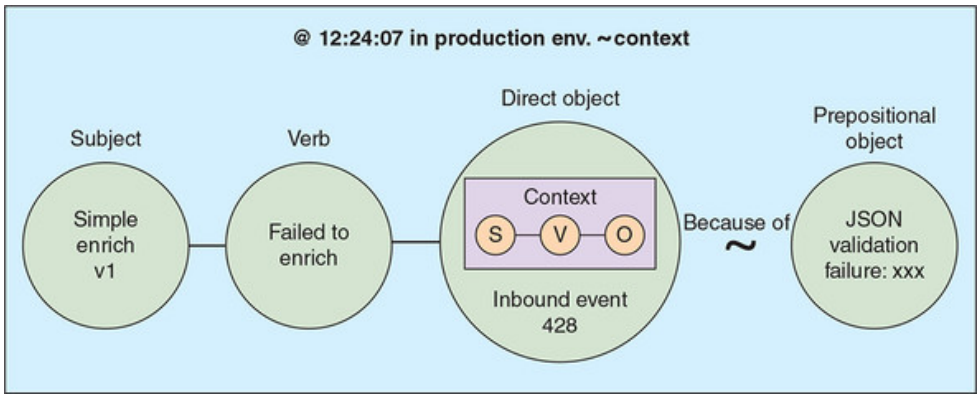
> *At 12:24:07 in our production environment, SimpleEnrich v1 failed to enrich Inbound Event 428 because it failed JSON Schema validation.*

Does this look familiar? It contains all the same grammatical components as the events we introduced in chapter 2:

- ***Subject***— In this case, our stream processing job, SimpleEnrich v1, is the entity carrying out the action of this event.
- ***Verb***— The action being done by the *subject* is, in this case, "failed to enrich."
- ***Direct object***— The entity to which the action is being done is Inbound Event 428.
- ***Timestamp***— This tells us exactly when this failure occurred.

We have another piece of *context* besides the timestamp: the environment in which the failure happened. Finally, we also have a *prepositional object*—namely, the reason that the enrichment failed. We call this *prepositional* because it is associated with the event via a prepositional phrase —in this case, "because of." Figure 8.5 clarifies the relationship between our event's constituent parts: *subject*, *verb*, *direct object*, *prepositional object* (the reason for the failure), and *context*.

Figure 8.5. We are representing our enrichment as an event: the subject is the enrichment job itself, the verb is "failed to enrich," and the direct object is the event we failed to enrich.

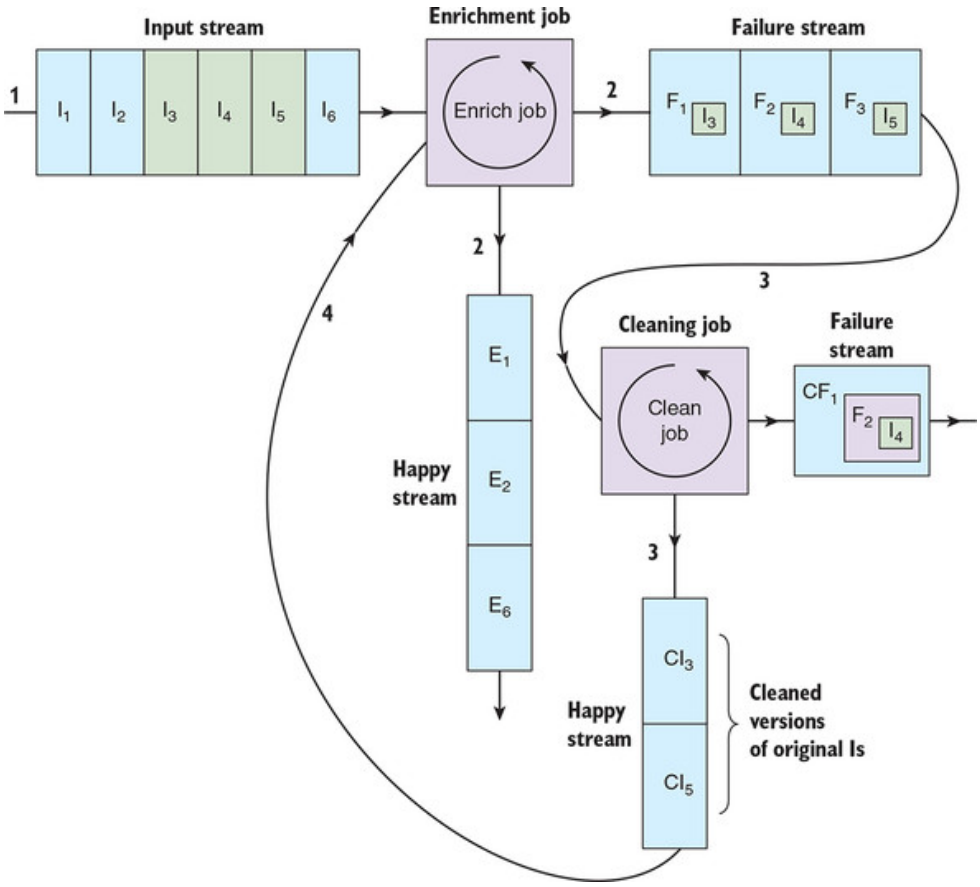


It may seem a little strange to have the direct object of our failure event be an event—specifically, the inbound event that failed enrichment. But this "turtles all the way down" design is powerful: it means that our failure event contains within it all the information required to retry the failed processing in the future. Without this design, we would have to manually correlate the failure messages with the original input events if we wanted to attempt reprocessing of failed events in the future.

This might sound a little theoretical; after all, the events have failed processing with an unrecoverable error once. What makes us think that this error might be recoverable in the future? In fact, there are lots of reasons that we shouldn't be binning this failed event just yet:

- Perhaps the enrichment failed because of a problem with a third-party service that has since been fixed. For example, the third-party service had an outage due to a denial-of-service attack, which has since been resolved.
- Perhaps some events failed enrichment because they were serialized with a version of a schema that somebody had forgotten to upload into our schema repository. Once this has been fixed and the schema uploaded, the events can be reprocessed.
- Perhaps a whole set of events failed enrichment because the source system had corrupted them all in a systemic way—for example, the source system had accidentally URL-encoded the customers' email addresses. We could write a quick stream processing job to read the corrupted events, fix them, and write them back to the original stream, ready for a second attempt at enrichment, as in figure 8.6.

Figure 8.6. Our enrichment job reads six input events; three fail enrichment and are written out to our failure stream. We then feed those three failures into a cleaning job that attempts to fix the corrupted events. The job manages to fix two events, which are fed back into the original enrichment job.



This last example is starting to hint at a key aspect of this design: its *composability* across multiple event streams within our unified log. We'll look at this next.

8.2.3. Composing our happy path across jobs

If our stream processing jobs follow the simple architecture set out previously, we can create a happy path composed of multiple stream processing jobs: the happy path output of one job is fed in as the input of the next job. Figure 8.7 illustrates this process.

Figure 8.7. We have composed a happy path by chaining together the happy output event stream of each stream processing job as the input stream of the next job.

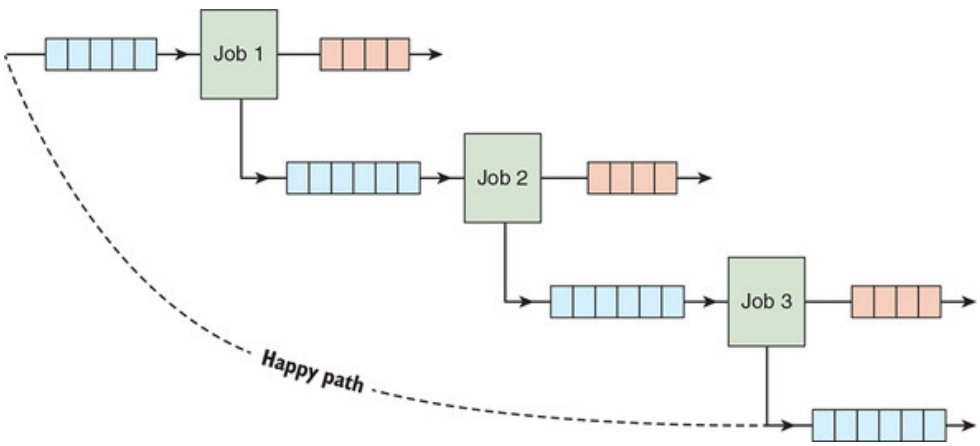


Figure 8.7 deliberately elides how we handle the failure paths of our individual jobs. How we handle the failure events emitted by a given job will

depend on a few factors:

- Do we expect to be able to recover from a job's failures in the future, and do we care enough to attempt recovery?
- How will we monitor error rates, and what constitutes an acceptable error rate for a given job?
- Where will we archive our failures?

These kinds of questions may well have different answers for different stream-processing jobs. For example, a job auditing customer refunds might take individual failures much more seriously than a job providing vanity metrics for a display panel in reception. But by modeling our failures as events, we should be able to create highly reusable failure-handling jobs, because they speak our *lingua franca* of well-structured events.

8.3. Failure composition with Scalaz

> *If you fail to plan, you are planning to fail!*
>
> *Benjamin Franklin*

So far, we have looked at the concept of the failure path at an architectural level, but how do we implement these patterns inside our stream processing jobs? It's time to add a new tag team to our unified log arsenal: Scala and Scalaz.

8.3.1. Planning for failure

Imagine that we are working at an online retailer that sells products in three currencies (euros, dollars, and pounds), but does all of this financial reporting in euros, its base currency. Our retailer already has all customer orders available in a stream in its unified log, so the managers want us to write a stream processing job that does the following:

- *Reads* customer orders from the incoming event stream
- *Converts* all customer orders into euros
- *Writes* the updated customer orders to a new event stream

The currency conversion will be done using the live (current) exchange rate, to keep things simple. Our job can look up the exchange rate by making an API call to a third-party service called Open Exchange Rates (https://openexchangerates.org/).

This sounds simple. What could go wrong with the operation of our job? In fact, a few things:

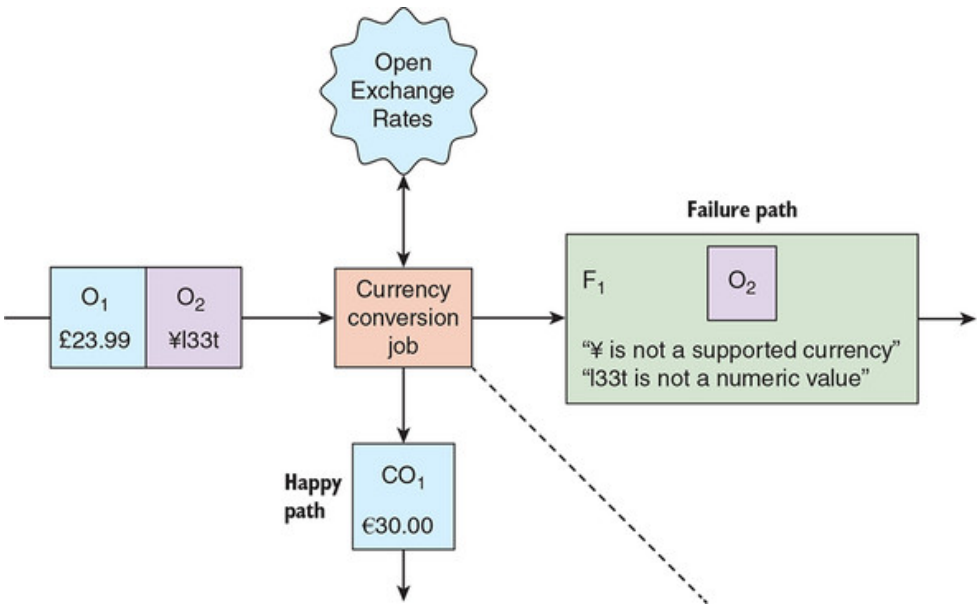
- Perhaps Open Exchange Rates is having an unplanned outage or planned downtime to facilitate an upgrade.
- Perhaps test data made it into the production flow, and the customer order is in a currency other than one of the three allowed ones.
- Perhaps a hacker has managed to transact an order with a non-numeric order value, getting themselves a huge flat screen TV for ¥l33t.

An interesting thing about these failures is that they are, as a management consultant might say, the opposite of *mutually exclusive, collectively exhaustive* (MECE):

- They are not *mutually exclusive*. We could have the order in the wrong currency and the order value be non-numeric.
- They are not *collectively exhaustive*. We could always experience a *novel error*. If we experience a novel error, we will add it to our failure handling, but there can always be novel errors.

Putting all this together, it is clear that our stream processing job will need to plan for failure: in addition to the output stream of orders in our base currency—our happy path—we will need a second output stream to report our currency conversion failures—our failure path. Figure 8.8 illustrates the overall job flow.

Figure 8.8. Our input event stream contains two customer order events. The valid one is successfully converted into euros and written to our happy stream. The second event is invalid and is written to our failure stream; our failure event records the failures encountered as well as the original event.



We are not particularly interested in the "plumbing" of this job; we have explored the mechanics of reading and writing to event streams in detail in previous chapters. This chapter is all about planning for failure—so the

important thing is learning how to assemble the internal logic of this job in a way that can cope with all these possible failures.

For these purposes, we can work with a stripped-down Scala application—just a few functions, unit tests, and a simple command-line interface.

8.3.2. Setting up our Scala project

Let's get started. We are going to create our Scala application by using Gradle. Scala Build Tool (SBT) is the more popular build tool for Scala, but we are familiar with Gradle and it works fine with Scala, so let's stick with that.

First, create a directory called forex, and then switch to that directory and run the following:

```
$ gradle init --type scala-library
...
BUILD SUCCESSFUL
...
```

As we did in chapter 3, we'll now delete the stubbed Scala files that Gradle created:

```
$ rm -rf src/*/scala/*
```

The default build.gradle file in the project root isn't quite what we need either, so replace it with the following code.

Listing 8.4. build.gradle

```
apply plugin: 'scala'                                                    1

repositories {
  mavenCentral()
  maven {                                                                2
    url 'http://oss.sonatype.org/content/repositories/releases'
  }
}

version = '0.1.0'

ScalaCompileOptions.metaClass.daemonServer = true                        3
ScalaCompileOptions.metaClass.fork = true
ScalaCompileOptions.metaClass.useAnt = true
ScalaCompileOptions.metaClass.useCompileDaemon = false

dependencies {                                                           4
```

```
    runtime 'org.scala-lang:scala-compiler:2.12.7'
    compile 'org.scala-lang:scala-library:2.12.7'
    compile 'org.scalaz:scalaz-core_2.12:7.2.27'
    testCompile 'org.specs2:specs2_2.12:3.8.9'
    testCompile 'org.typelevel:scalaz-specs2_2.12:0.5.2'
  }

  task repl(type:JavaExec) {                                    5
      main = "scala.tools.nsc.MainGenericRunner"
      classpath = sourceSets.main.runtimeClasspath
      standardInput System.in
      args '-usejavacp'
  }
```

- ***1*** **We use Gradle's Scala plugin for projects involving Scala code.**
- ***2*** **Some of our dependencies are published to Sonatype.**
- ***3*** **So that Gradle can compile Scala against Java 8**
- ***4*** **For our runtime, we need Scala and scalaz; for testing, we will use Specs2.**
- ***5*** **Task to start the Scala console**

With that updated, let's check that everything is still functional:

```
$ gradle build
...
BUILD SUCCESSFUL
...
```

Okay, good—now let's write some Scala code!

8.3.3. From Java to Scala

Remember that we need to check whether our customer order is one of our three supported currencies (euros, dollars, or pounds).

If we were still working in Java, we might write a function that threw a checked exception if the currency was not in one of our supported currencies. In the following listing, we have a function that parses the currency string:

- If it is a supported currency, the function returns our currency as a Java enum.
- Otherwise, it throws a checked exception, `UnsupportedCurrencyException`.

Listing 8.5. CurrencyValidator.java

```
package forex;

import java.util.Locale;

public class CurrencyValidator {
  public enum Currency {USD, GPB, EUR}                    1

  public static class UnsupportedCurrencyException
    extends Exception {                                   2
    public UnsupportedCurrencyException(String raw) {
      super("Currency must be USD/EUR/GBP, not " + raw);
    }
  }

  public static Currency validateCurrency(String raw)
    throws UnsupportedCurrencyException {                 3

    String rawUpper = raw.toUpperCase(Locale.ENGLISH);
    try {
      return Currency.valueOf(rawUpper);
    } catch (IllegalArgumentException iae) {              4
      throw new UnsupportedCurrencyException(raw);
    }
  }
}
```

- ***1*** **Defines our three supported currencies as a Java enum**
- ***2*** **A checked exception for unsupported currencies**
- ***3*** **Our validation function throws our checked exception if the currency is unsupported.**
- ***4*** **We catch the unchecked exception and convert it into our checked exception.**

One of the nice aspects of Scala for recovering Java programmers is that it's possible to port existing Java code to Scala code (albeit unidiomatic Scala code) with only cosmetic syntactical changes. The following listing contains just such a direct, unidiomatic port of our existing `CurrencyConverter` class to a Scala object.

Listing 8.6. currency.scala

```
package forex

object Currency extends Enumeration {                     1
  type Currency = Value
  val Usd = Value("USD")
  val Gbp = Value("GBP")
  val Eur = Value("EUR")
}
```

```
case class UnsupportedCurrencyException(raw: String)
  extends Exception("Currency must be USD/EUR/GBP, not " + raw)     2

object CurrencyValidator1 {                                        3

  @throws(classOf[UnsupportedCurrencyException])
  def validateCurrency(raw: String): Currency.Value = {

    val rawUpper = raw.toUpperCase(java.util.Locale.ENGLISH)
    try {
      Currency.withName(rawUpper)                                  4
    } catch {
      case nsee: NoSuchElementException =>                         5
        throw new UnsupportedCurrencyException(raw)
    }
  }
}
```

- ***1* Our Java enum is now a Scala object extending Enumeration.**
- ***2* Our Java exception is now a Scala case class.**
- ***3* Our Java class is now a Scala object.**
- ***4* Last expression evaluated is returned without needing return statement**
- ***5* Catching exceptions in Scala uses pattern-matching semantics.**

If you are completely new to Scala, here are a few immediate things to note about this code compared to Java:

- A single Scala file can contain multiple independent classes and objects—in which case, we give the file a descriptive name starting with a lowercase letter.
- No need for semicolons at the end of each line, though they are still useful to separate multiple statements on the same line.
- We use `val` to assign what in Java would be variables. The internal state of a `val` can be modified, but a `val` cannot be reassigned, so no `val a = 0; a = a + 1`.
- In place of a class containing a static method, we now have a *Scala object* containing a method. An object in Scala is a singleton, a unique instance of a class.
- A function definition starts with the keyword def.
- Scala has type inference, which means that it has the ability to figure out the types that are left off.

There is an interesting difference in the behavior of our `validateCurrency` function too: our function now has the `@throws` annotation recording the exception that it may throw. Scala doesn't distinguish between checked and unchecked exceptions, so this annotation is optional, and useful only if some Java code is going to call this function.

Another nice difference between Scala and Java is the interactive console, or *read-eval-print loop* (REPL), available with Scala. Let's use this to put our new Scala code through its paces. From the project root, start up the REPL with this command:

```
$ gradle repl --console plain
...
scala>
```

The Scala prompt is waiting for you to type in a Scala statement to execute. Let's try this one:

```
scala> forex.CurrencyValidator1.validateCurrency("USD")
res0: forex.Currency.Value = USD
```

The second line shows you the output from the validateCurrency function: it's returning a USD-flavored value from our Currency enumeration. Let's check that the case-insensitivity is working too:

```
scala> forex.CurrencyValidator1.validateCurrency("eur")
res1: forex.Currency.Value = EUR
```

That seems to be working. Now let's see what happens if we pass in an invalid value:

```
scala> forex.CurrencyValidator1.validateCurrency("dogecoin")
forex.UnsupportedCurrencyException: Currency must be USD/EUR/GBP, not doc
  at forex.CurrencyValidator1$.validateCurrency(currency.scala:23)
  ... 28 elided
```

That looks correct. Trying to validate `dogecoin` as a currency is throwing an `UnsupportedCurrencyException`. We have ported our Java currency validator to Scala—but so far, we have only tweaked syntax; the semantics of our failure handling are the same. We can do better, as you'll see in the next iteration.

8.3.4. Better failure handling through Scalaz

Let's take a second pass at our currency validator. You can see the updated object, `CurrencyValidator2`, in the following listing. We haven't changed our `Currency` enumeration, and are no longer using the `UnsupportedCurrencyException`, so these are left out.

Listing 8.7. CurrencyValidator2.scala

```
package forex

import scalaz._                                                    1
import Scalaz._                                                    1

object CurrencyValidator2 {


  def validateCurrency(raw: String):
    Validation[String, Currency.Value] = {                         2

    val rawUpper = raw.toUpperCase(java.util.Locale.ENGLISH)
    try {
      Success(Currency.withName(rawUpper))                         3
    } catch {
      case nsee: NoSuchElementException =>
        Failure("Currency must be USD/EUR/GBP and not " + raw)     4
    }
  }
}
```

- ***1*** **Imports the Scalaz library**
- ***2*** **Our return type is a Scalaz Validation "box" around either a String or a Currency.**
- ***3*** **On success, we return our Currency inside a Success-flavored Validation.**
- ***4*** **On failure, we return our error String inside a Failure-flavored Validation.**

Again, here are a few notes to make the following listing a little easier to digest:

- This time, our file contains only a single object, so our filename matches the object.
- We no longer throw an exception so have no further need for the `@throws` annotation.
- Scala uses `[ ]` for specifying generics, whereas Java uses <>.

Most important, in place of throwing exceptions, we have moved our failure path into the return value of our function: this complex-looking `Validation` type, provided by Scalaz. Let's see how this `Validation` type behaves in the REPL; note that we have to quit and restart the console to force a recompile:

```
<Ctrl-D>
$ gradle repl --console plain
...
scala> forex.CurrencyValidator2.validateCurrency("eur")
```

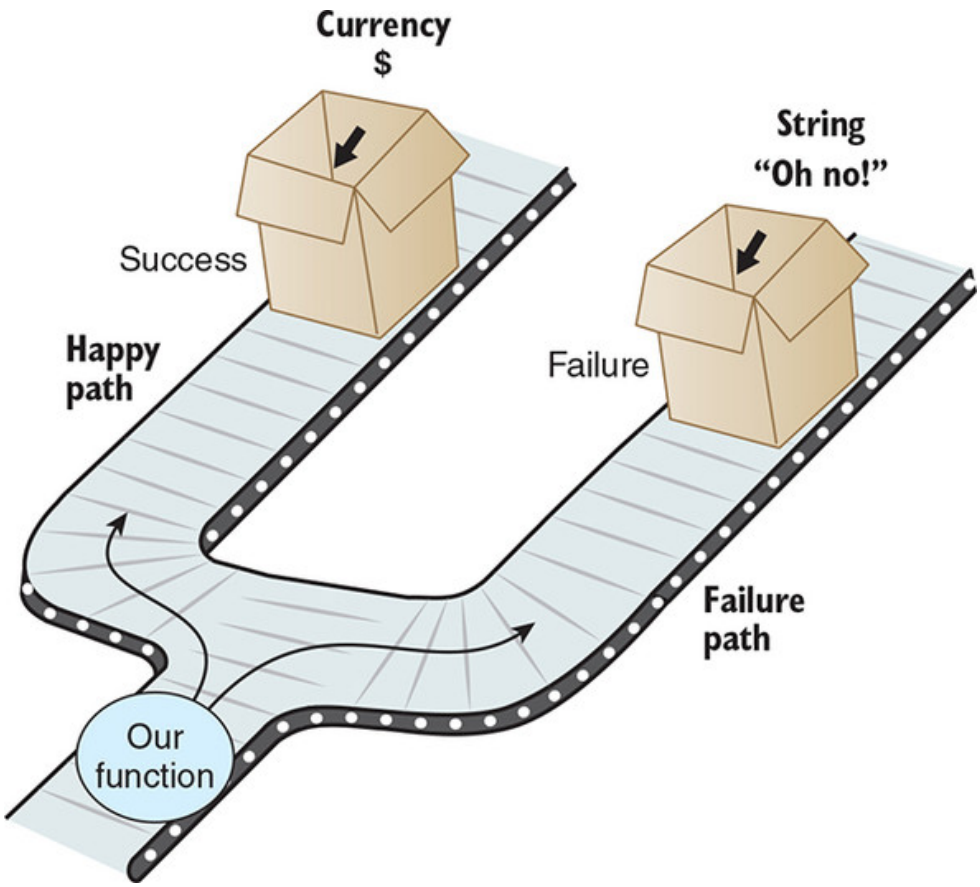
```
res0: scalaz.Validation[String,forex.Currency.Value] = Success(EUR)

scala> forex.CurrencyValidator2.validateCurrency("dogecoin")
res1: scalaz.Validation[String,forex.Currency.Value] =
 Failure(Currency must be USD/EUR/GBP and not dogecoin)
```

You can think of `Validation` as a kind of box, or context, that describes whether the value inside it is on our happy path (called `Success` in Scalaz) or our failure path (called `Failure`). Even neater, the value inside the `Validation` box can have a different *type* on the `Success` side versus on the `Failure` side:

```
Validation[String, Currency.Value]
```

The first type given, `String`, is the type that we will use for our failure path; the second type, `Currency.Value`, is the type that we will use for our success path. Figure 8.9 shows how `Validation` is working, using the metaphor of cardboard boxes and paths.

Figure 8.9. For the happy path, our function returns a `Currency` boxed inside a `Success`. For the failure path, our function returns an error `String` boxed inside a `Failure`. `Success` and `Failure` are the two modes of the Scalaz `Validation` type.



This switch from using exceptions to using Scalaz's `Validation` to box either success or failure might not seem like a big change, but it's going to be our key building block for working with failure in Scala.

8.3.5. Composing failures

We're now happy with our function to validate that the incoming currency is one of our three supported currencies. But remember, another possible bug remains in our event stream:

> *Perhaps a hacker has managed to transact an order with a non-numeric order value, getting themselves a huge flat-screen TV for ¥l33t.*

We can put a stop to this with an `AmountValidator` object containing another validation function: one that checks that the incoming stringly typed order amount can be parsed to a Scala `Double`. This is set out in the following listing.

Listing 8.8. AmountValidator.scala

```
package forex

import scalaz._
import Scalaz._

object AmountValidator {

  def validateAmount(raw: String): Validation[String, Double] = {     1

    try {
      Success(raw.toDouble)                                           2
    } catch {
      case nfe: NumberFormatException =>
        Failure("Amount must be parseable to Double and not " + raw)  3
    }
  }
}
```

- ***1* Our return type is another Validation "box," around either a String or a Double.**
- ***2* On success, we return our Double inside a Success.**
- ***3* On failure, we return our error String inside a Failure.**

By now, the pattern should be familiar: our function uses a `Validation` to represent our return value being either on the happy path (with a `Success`), or on the failure path (with a `Failure`). As before, we are returning different types of values inside our `Success` and our `Failure`: in this case, a `Double` or a `String`, respectively.

A quick check to make sure this is working in the Scala REPL:

```
<Ctrl-D>
$ gradle repl --console plain
...
scala> forex.AmountValidator.validateAmount("31.98")
res2: scalaz.Validation[String,Double] = Success(31.98)

scala> forex.AmountValidator.validateAmount("L33T")
res3: scalaz.Validation[String,Double] = Failure(Amount must be
 parseable to Double and not L33T)
```

Great! So we now have two validation functions:

- `CurrencyValidator2.validateCurrency`
- `AmountValidator.validateCurrency`

Both functions would have to return a `Success` in order for us to assemble a valid order total. If we get a `Failure`, or indeed two `Failures`, then we will find ourselves squarely on the failure path. Let's build a function now that attempts to construct an order total by running both of our validation functions. See the following listing for the code.

Listing 8.9. OrderTotal.scala

```
package forex

import scalaz._
import Scalaz._

case class OrderTotal(currency: Currency.Value, amount: Double)    1

object OrderTotal {

  def parse(rawCurrency: String, rawAmount: String):
    Validation[NonEmptyList[String], OrderTotal] = {               2

    val c = CurrencyValidator2.validateCurrency(rawCurrency)       3
    val a = AmountValidator.validateAmount(rawAmount)              4

    (c.toValidationNel |@| a.toValidationNel) {                    5
      OrderTotal(_, )
    }
  }
}
```

- ***1* A case class representing our order total**
- ***2* Our parse method returns a Scalaz Validation.**
- ***3* Validates the currency and assigns to a val**
- ***4* Validates the amount and assigns to a val**

- ***5*** **Combines our two validations into the return value for this function**

A lot of new things are certainly happening in a small amount of code, but don't worry, we will go through this listing carefully in a short while. Before we do that, let's return to the Scala REPL and see how this `parse` function performs, first if both the currency and amount are valid:

```
<Ctrl-D>
$ gradle repl --console plain
...
scala> forex.OrderTotal.parse("eur", "31.98")
res0: scalaz.Validation[scalaz.NonEmptyList[String],forex.OrderTotal]
 = Success(OrderTotal(EUR,31.98))
```

This result makes sense: because both validations passed, we are staying on the happy path, and our `OrderTotal` is boxed in a `Success` to reflect this. Now let's see what happens if one or either of our validations fails:

```
scala> forex.OrderTotal.parse("dogecoin", "31.98")
res2: scalaz.Validation[scalaz.NonEmptyList[String],forex.OrderTotal]
 = Failure(NonEmpty[Currency must be USD/EUR/GBP and not dogecoin])

scala> forex.OrderTotal.parse("eur", "L33T")
res3: scalaz.Validation[scalaz.NonEmptyList[String],forex.OrderTotal]
 = Failure(NonEmpty[Amount must be parseable to Double and not L33T])
```

In both cases, we end up with a `Failure` containing a `NonEmptyList`, which in turn contains our error message as a `String`. In fact, `NonEmptyList` is another Scalaz type. It's similar to a standard Java or Scala `List`, except that a `NonEmptyList` (sometimes called a Nel, or NEL, for short) cannot be, well, empty. This suits our purposes well; if we have a `Failure`, we know at least one cause of it, and so our list of error messages will never be empty.

Why do we need a NEL on the `Failure` side in the first place? Hopefully, this next test should make the reason clear:

```
scala> forex.OrderTotal.parse("dogecoin", "L33T")
res4: scalaz.Validation[scalaz.NonEmptyList[String],forex.OrderTotal]
 = Failure(NonEmpty[Currency must be USD/EUR/GBP and not dogecoin,
 Amount must be parseable to Double and not L33T])
```

It's a little bit long-winded, but you can see that the value returned from the parse function is now a `Failure` dutifully recording both error messages in `String` form; this approach is similar to the way you might see

multiple validation errors (Phone Is Missing Country Code, and so forth) on a website form when you click the Submit button.

Now that you understand what the function does, let's return to the code itself:

```
(c.toValidationNel |@| a.toValidationNel) {
  OrderTotal(_, _)
}
```

What exactly is going on here, and what is that mysterious `|@|` operator doing? The first thing to explain is the `toValidationNel` method calls. There is not much mystery here: this is a method available to any Scalaz `Validation` that will promote the `Failure` value into a NEL. Here's a quick demonstration in the Scala REPL:

```
scala> import scalaz._
import scalaz._

scala> import Scalaz._
import Scalaz._

scala> val failure = Failure("OH NO!")
failure: scalaz.Failure[String] = Failure(OH NO!)

scala> failure.toValidationNel
res8: scalaz.ValidationNel[String,Nothing] = Failure(NonEmpty[OH NO!])
```

See how the error message is now inside a NEL? This happens only in the case of `Failure`. If we have a `Success`, the value inside is untouched, although the overall type of the `Validation` will still change:

```
scala> val success = Success("WIN!")
success: scalaz.Success[String] = Success(WIN!)

scala> success.toValidationNel
res6: scalaz.ValidationNel[Nothing,String] = Success(WIN!)
```

Now onto the `|@|` operator. Unfortunately, there is no official Scalaz documentation about this, but if you dig into the code, you will find it referred to as follows:

> *[a] DSL for constructing Applicative expressions*

The `|@|` operator is sometimes referred to as the *Home Alone* or *chelsea bun* operator, but we prefer to call it the *Scream* operator after the Munch painting. Regardless, this operator is doing something clever: it is compos-

ing our two input `Validations` into a new output `Validation`, intelligently determining whether the output `Validation` should be a `Success` or a `Failure` based on the inputs. Figure 8.10 shows the different options.

Figure 8.10. Applying the Scream operator to two Scalaz `Validations` gives us a matrix of possible outputs, depending on whether each input `Validation` is a `Success` or `Failure`. Note that we get an output of `Success` only if both inputs are `Success`.

As you can see in figure 8.10, the Scream operator allows us to compose multiple `Validation` inputs into a single `Validation` output. The example uses two inputs just to keep things simple: we could just as easily compose nine or twenty `Validation` inputs into a single `Validation` output. If we composed twenty `Validation` inputs with the Scream operator, *all* of our twenty inputs must be a `Success` for us to end up with a `Success` output. On the other hand, if all of our twenty inputs were `Failures` containing a single error `String`, then our output would be a `Failure` containing a `NonEmptyList` of twenty error `Strings`.

You have seen how to handle failure inside a single processing step: we can perform multiple pieces of validation in parallel, and then compose the successes or failures into a single output. If this is failure processing *in the small*, how do we start to handle failures between different processing steps inside the same job? We'll look at this next. Feel free to grab a cup of coffee before you apply what you've learned so far on our unified log.

8.4. Implementing railway-oriented processing

> *Everything has an end, and you get to it if you only keep all on.*
>
> *E. Nesbit, The Railway Children*

A common theme running through this chapter has been the idea of our code embarking on a happy path and dropping down to a failure path if something (or multiple things) goes wrong. We have looked at this pattern at a large scale, across multiple stream processing jobs, as well as at a small scale, at the level of composing multiple failures inside a single processing step. Now let's look at the middle ground: handling failure across multiple processing steps inside a single job.

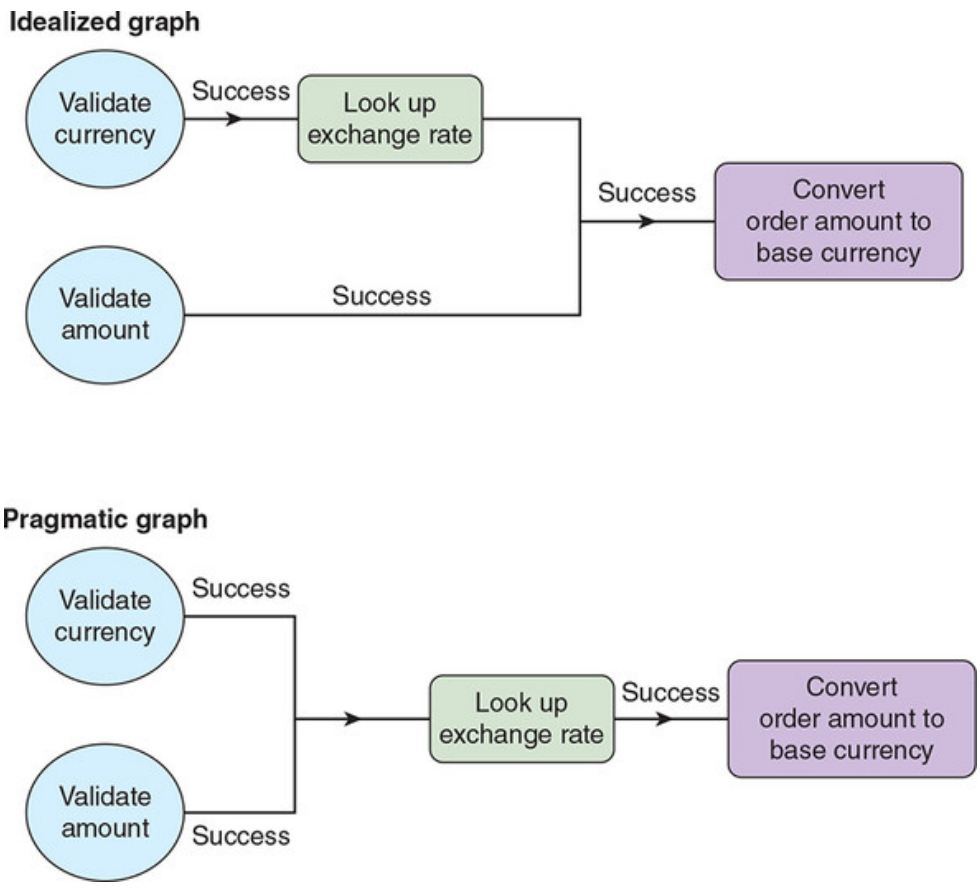
8.4.1. Introducing railway-oriented processing

In the preceding section, the Scream operator let us compose an output from separate pieces of processing into a single unified output. Specifically, we composed the `Validation` outputs from our currency and amount parsing into a single `Validation` output. Where do we go from here?

Remembering back to our original brief, we also need the current exchange rate between our order's currency and our employer's base currency. Again, this requirement could also take us onto the failure path: fetching the exchange rate could fail for various reasons. We have several processing steps now, so let's sketch out an end-to-end happy path that incorporates all of this processing. Figure 8.11 presents two separate versions of this happy path:

- The *idealized happy path*—This expresses the dependencies between each piece of work. Specifically, we need a successfully validated currency to look up the currency, but we don't need the validated amount until we are trying to perform the conversion.
- The *pragmatic happy path*—We look up the currency only if both the currency and the amount validate successfully.

Figure 8.11. The idealized and pragmatic happy paths vary based on how late the exchange rate lookup is performed. In the pragmatic happy path, we validate as much as we can before attempting the exchange rate lookup.



The pragmatic happy path is so-called because looking up a currency from a third-party service's API over HTTP is an *expensive* operation,

whereas validating our order amount is relatively cheap. We could be performing this processing job for many millions of events, so even small unnecessary delays will add up; therefore, if we can fail fast and save ourselves some pointless currency lookups, we should.

Figure 8.11 shows us only the end-to-end happy path, resulting in a customer order successfully converted into the retailer's base currency. Figure 8.12 provides the next level of detail, imagining our processing job as a railway line (or conveyor belt), with two tracks:

- ***The happy track***— This shows the type transformations that occur as we successfully validate first the currency and amount (one processing step), then successfully look up the exchange rate (our second processing step), and finally convert the order amount.
- ***The failure track***— We can be switched onto this track if any of our processing steps fail.

This is not my own metaphor: *railway-oriented programming* was coined by functional programmer Scott Wlaschin in his eponymous blog post (https://fsharpforfunandprofit.com/posts/recipe-part2/). Scott's blog post uses the railway metaphor with happy and failure paths to introduce compositional failure handling in the F# programming language. Another functional language, F# enables a similar approach to failure processing to our Scala-plus-Scalaz combination. We strongly recommend reading Scott's post after you have finished this chapter; in the meantime, here is railway-oriented programming in Scott's own words:

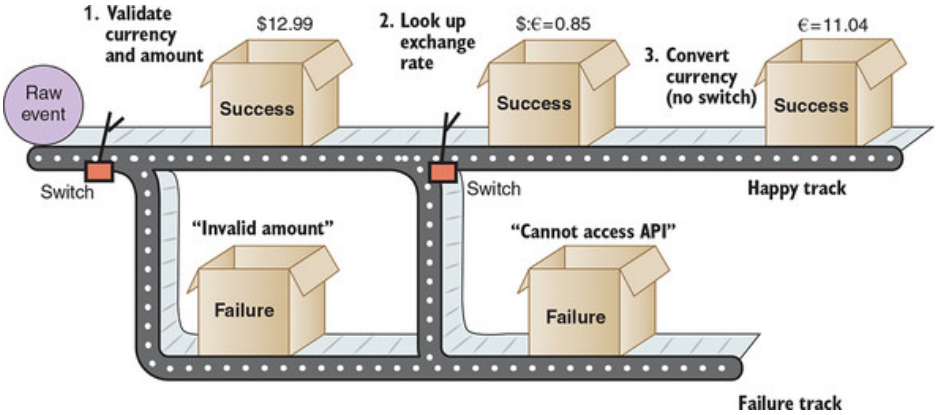
> *What we want to do is connect the* `Success` *output of one to the input of the next, but somehow bypass the second function in case of a* `Failure` *output....There is a great analogy for doing this—something you are probably already familiar with. Railways! Railways have switches ("points" in the UK) for directing trains onto a different track. We can think of these "Success/Failure" functions as railway switches....We will have a series of black-box functions that appear to be straddling a two-track railway, each function processing data and passing it down the track to the next function....Note that once we get on the failure path, we never (normally) get back onto the happy path. We just bypass the rest of the functions until we reach the end.*

The railway-oriented approach has important properties that might not be immediately obvious but affect how we use it:

- It *composes* failures within an individual processing step, but it *fails fast* across multiple steps. This makes sense: if we have an invalid currency code, we can validate the order amount as well (same step), but

we can't proceed to getting an exchange rate for the corrupted code (the next step).

- The type inside our `Success` can change between each processing step. In figure 8.12, our `Success` box contains first an `OrderTotal`, then an exchange rate, and then finally an `OrderTotal` again.
  Figure 8.12. Processing of our raw event proceeds down the happy track, but a failure at any given step will switch us onto the failure track. When we are on the failure track, we stay there, bypassing any further happy-track processing.



- The type inside our Failure must stay the same between each processing step. Therefore, we have to choose a type to record our failures and stick to it. We have used a `NonEmptyList[String]` in this chapter because it is simple and flexible.

Enough of the theory for now—let's build our railway in Scala.

8.4.2. Building the railway

First, we need a function representing our exchange-rate lookup, which could fail. Because our focus in this chapter is on failure handling, our function will contain some hardcoded exchange rates plus some hard-wired "random failure." If you are interested in looking up real exchange rates in Open Exchange Rates, you can always check out Snowplow's scala-forex project (https://github.com/snowplow/scala-forex/).

The code for our new function, `lookup`, is in the following listing. The function consists of a single pattern-match expression that returns the following:

- On the happy track, an exchange rate `Double` boxed in a `Success`
- On the failure track, an error `String` boxed in a `Failure`

Listing 8.10. ExchangeRateLookup.scala

```
package forex

import util.Random
```

```
import scalaz._
import Scalaz._

object ExchangeRateLookup {

  def lookup(currency: Currency.Value):
    Validation[String, Double] = {                              1

    currency match {
      case Currency.Eur     => 1D.success                       2
      case _ if isUnlucky() => "Network error".failure          3
      case Currency.Usd     => 0.85D.success
      case Currency.Gbp     => 1.29D.success
      case _                => "Unsupported currency".failure   4
    }
  }

  private def isUnlucky(): Boolean =
    Random.nextInt(5) == 4
}
```

- ***1* Returns either a Failure String or a Success Double (the exchange rate)**
- ***2* .success and .failure are Scalaz "sugar" for Success() and Failure().**
- ***3* Simulates a network error when retrieving USD or GBP exchange rate**
- ***4* Should never happen, but we want an exhaustive pattern match**

If you haven't seen a pattern match before, you can think of it as similar to a C- or Java-like switch statement, but much more powerful. We can match against specific cases such as `Currency.Eur`, and we can also perform wildcard matches using _. The `if isUnlucky()` is called a *guard*; it makes sure that we match this pattern only if the condition is met. As you can see from the definition of the `isUnlucky` function, we are simulating a network failure that should happen 20% of the time when we are looking up the exchange rate over the network.

An important point on the final _ wildcard match: we include this to ensure that our pattern match is *exhaustive*—that all possible patterns that could occur are handled. Without this final wildcard match, the code would still compile, but we are sitting on a potential problem if the following occur:

1. A new currency (for example, Yen) is added to the `Currency` enumeration.
2. Support for the new currency is added to the `validateCurrency` function.

3. But we forget to add this new currency into this pattern-match statement.

In this case, the pattern match would throw a runtime `MatchError` on every yen lookup, causing our stream processing job to crash. We want to avoid this, so our final _ wildcard match defends against this *pre-bug*, a bug that has not happened yet.

Let's put our new function through its paces in the Scala REPL:

```
<Ctrl-D>
$ gradle repl --console plain
...
scala> import forex.{ExchangeRateLookup, Currency}
import forex.{ExchangeRateLookup, Currency}

scala> ExchangeRateLookup.lookup(Currency.Usd)
res2: scalaz.Validation[String,Double] = Success(0.85)

scala> ExchangeRateLookup.lookup(Currency.Gbp)
res2: scalaz.Validation[String,Double] = Success(1.29)
```

Great—that's all working fine. Our network connection is also suitably unreliable; every so often when looking up a rate, you should see an error like this:

```
scala> ExchangeRateLookup.lookup(Currency.Gbp)
res1: scalaz.Validation[String,Double] = Failure(Network error)
```

Now, remembering back to section 8.3, we have two functions that represent the two distinct steps in our processing, either of which could fail. Here are the signatures of both functions:

- `OrderTotal.parse(rawCurrency: String, rawAmount: String): Validation[NonEmptyList[String], OrderTotal]`
- `ExchangeRateLookup.lookup(currency: Currency.Value): Validation[String, Double]`

Note that the types on our failure track are not quite the same: a `NonEmptyList` of `String`s, versus a singular `String`. In theory, this sounds like it breaks the rule that "every `Failure` must contain the same type," but in practice it is easy to promote a single `String` to be a `NonEmptyList` of `String`s.

Remember that we want to try to generate an order total first, and only then look up our exchange rate. Let's create a new function,

`OrderTotalConverter1.convert`, which captures all of the processing we need to do in this job. The signature of this function should look like this:

```
convert(rawCurrency: String, rawAmount: String): ValidationNel[String,
     OrderTotal]
```

You haven't seen `ValidationNel[A, B]` before. This is Scalaz shorthand for `Validation[NonEmptyList[A], B]`. Putting this all together: we will attempt to generate an `OrderTotal` in our base currency from an incoming raw currency and order total. If we somehow end up on the failure path, we will return a `NonEmptyList` of the error `String`s that we encountered. Let's see an implementation of this function in the following listing.

Listing 8.11. OrderTotalConverter1.scala

```
package forex

import scalaz._
import Scalaz._

import forex.{OrderTotal => OT}                                    1
import forex.{ExchangeRateLookup => ERL}

object OrderTotalConverter1 {

  def convert(rawCurrency: String, rawAmount: String):
    ValidationNel[String, OrderTotal] = {                          2

    for {
      total <- OT.parse(rawCurrency, rawAmount)                    2
      rate  <- ERL.lookup(total.currency).toValidationNel          3
      base   = OT(Currency.Eur, total.amount * rate)               4
    } yield base
  }
}
```

- ***1* Aliases to make our function more readable**
- ***2* Parse our raw currency and amount into a Validation.**
- ***3* Look up our exchange rate into a Validation.**
- ***4* Calculate our order total in the base currency.**

There is some interesting new syntax here; that `for {} yield` is clearly not your grandfather's `for` loop! Before we dive into this, let's fire up the Scala REPL one last time for this chapter and put this function through its paces. First let's stick to the happy path (assuming our unreliable network lets us):

```
<Ctrl-D>
$ gradle repl --console plain
...
scala> forex.OrderTotalConverter1.convert("usd", "12.99")
res1: scalaz.ValidationNel[String,forex.OrderTotal]
 = Success(OrderTotal(EUR,11.0415))

scala> forex.OrderTotalConverter1.convert("EUR", "28.98")
res2: scalaz.ValidationNel[String,forex.OrderTotal]
 = Success(OrderTotal(EUR,28.98))
```

Now let's see about the failure path:

```
scala> forex.OrderTotalConverter1.convert("yen", "l33t")
res3: scalaz.ValidationNel[String,forex.OrderTotal]
 = Failure(NonEmptyList(Currency must be USD/EUR/GBP and not yen,
 Amount must be parseable to Double and not l33t))

scala> forex.OrderTotalConverter1.convert("gbp", "49.99")
res56: scalaz.ValidationNel[String,forex.OrderTotal]
 = Failure(NonEmptyList(Network error))
```

You might have to repeat the second conversion a few times to see the Network Error `Failure` caused by the exchange rate lookup. Also, remember that you will see the network error only if the raw currency and amount are valid: if either is invalid, we have already failed fast, and the exchange rate will not be looked up.

So our `convert()` function is working, but how is it working? The key is to understand the `for {} yield` construct. Wherever you see the `for` keyword in Scala, you are looking at a Scala `for` comprehension, which is syntactic sugar over a set of Scala's functional operations: `foreach`, `map`, `flatMap`, `filter`, or `withFilter`.[3] This is modeled on the `do` notation found in Haskell, a pure functional language.

3

More information on how the Scala yield keyword works can be found at https://docs.scala-lang.org/tutorials/FAQ/yield.html.

Scala will translate our `for {} yield` into a set of `flatMap` and `map` operations, as shown in listing 8.12. The code in `OrderTotalConverter2` is functionally identical to our previous `OrderTotalConverter1`; you can test it at the Scala REPL if you like:

```
<Ctrl-D>
$ gradle repl --console plain
```

```
...
scala> forex.OrderTotalConverter2.convert("yen", "l33t")
res3: scalaz.ValidationNel[String,forex.OrderTotal]
 = Failure(NonEmpty[Currency must be USD/EUR/GBP and not yen,
 Amount must be parseable to Double and not l33t])
```

Listing 8.12. OrderTotalConverter2.scala

```
package forex

import scalaz._
import Scalaz._
import scalaz.Validation.FlatMap._

import forex.{OrderTotal => OT}
import forex.{ExchangeRateLookup => ERL}

object OrderTotalConverter2 {

  def convert(rawCurrency: String, rawAmount: String):
    ValidationNel[String, OrderTotal] = {

    OT.parse(rawCurrency, rawAmount).flatMap(total =>                    1
      ERL.lookup(total.currency).toValidationNel.map((rate: Double) =>   2
        OT(Currency.Eur, total.amount * rate)))                          3
  }
}
```

- ***1*** **We replace our first <- with a .flatMap.**
- ***2*** **We replace our second <- with a .map.**
- ***3*** **No need to assign a name like "base" here**

Putting them side by side, `OrderTotalConverter1` is much more readable than `OrderTotalConverter2`, but this second version gives us a better starting point for understanding how we have implemented this fail-fast approach across multiple processing steps. This is the last piece of the railway-oriented processing puzzle.

Also note that `OrderTotalConverter1` does work only when compiled against Scala 2.11, as Scala 2.12 has introduced enhanced type checking that made type inference inside `for` comprehensions more difficult to achieve.

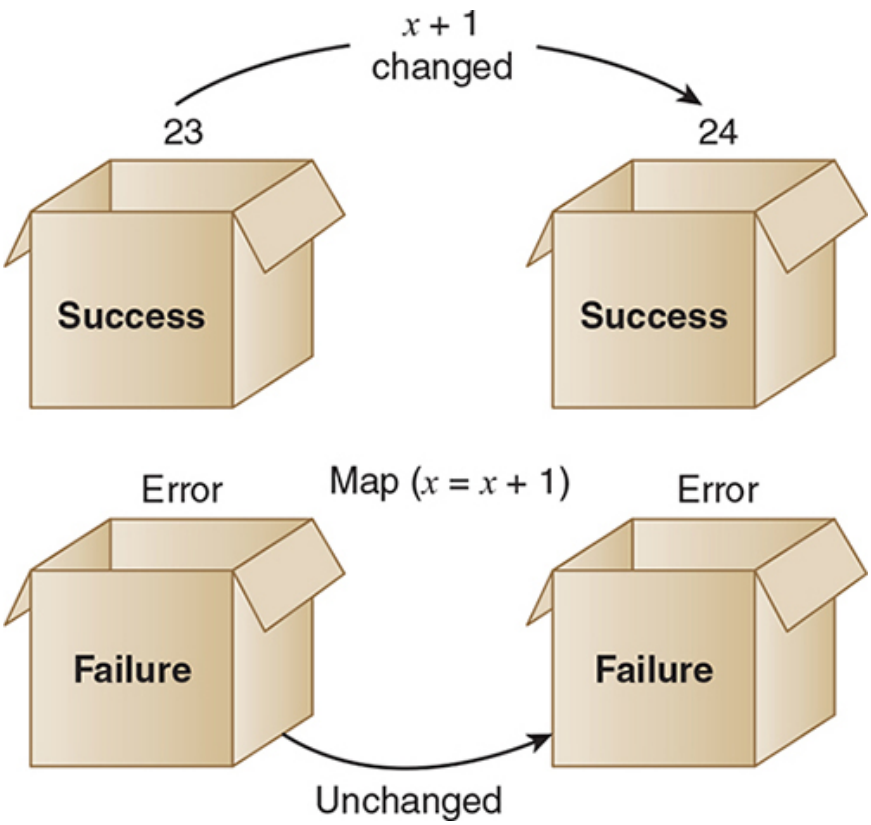
`flatMap` and `map` are the last two pieces in our toolkit for this chapter, so we'll go through these in some detail. We'll start with `map` as it's simpler to understand. Here is a simplified definition for the `map` method on a `Validation[F, S]` called `self`:

```
def map(aFunc: S => T): Validation[F, T] = self match {
  case Success(aValue) => Success(aFunc(aValue))
  case Failure(fValue) => Failure(fValue)
}
```

Because this is Scala, we are able to define `map` in terms of a single pattern-match expression. The way to read `aFunc: S => T` is as a function that takes an argument of type `S` and returns a value of type `T`. If our `Validation` is a `Success`, we apply the function supplied to `map` to the value contained inside the `Success`, resulting in a new value, possibly of a new type, but still safely wrapped in our `Validation` container. To put it another way: if we start with `Success[S]` and apply a function `S => T` to the value inside `Success`, we end up with `Success[T]`. On the other hand, if our `Validation` is a `Failure`, we leave this as is: a `Failure[F]` stays a `Failure[F]`, containing the exact same value.

Mapping over a `Validation` is visualized in figure 8.13, for both a `Success` and a `Failure`.

Figure 8.13. If we map a simple function (adding 1 to a number) to a `Success`-boxed 23 and a `Failure`-boxed error `String`, the `Failure` box is untouched, but the `Success` box now contains 24.



Now let's take a look at `flatMap`. Like `map`, `flatMap` takes a function, applies it to a `Validation[F, S]`, and returns a `Validation[F, T]`. The difference is in the function that `flatMap` takes as its argument: the function has the type `S => Validation[F, T]`. In other words, `flatMap` expects a function that takes a value and produces a new value inside a new

`Validation` container. You may be thinking, hang on a moment—why does a `flatMap`, like a `map`, return this

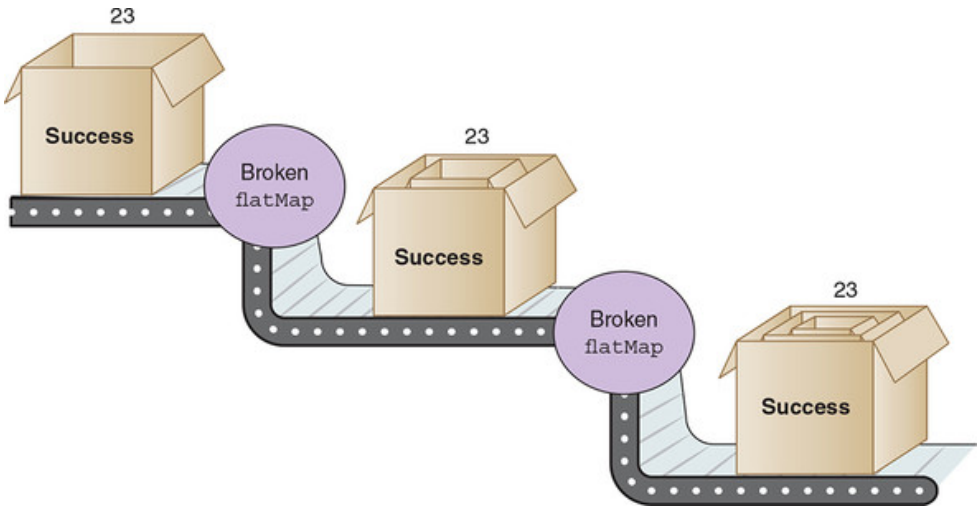
```
Validation[F, T]
```

and not this:

```
Validation[F, Validation[F, T]], given the supplied function
```

The answer to this question lies in the *flat* in the name: `flatMap` *flattens* the two `Validation` containers into one. `flatMap` is the secret sauce of our railway-oriented approach: it allows us to chain multiple processing steps together into one stream processing job without accumulating another layer of `Validation` boxing at every stage. Figure 8.14 depicts the unworkable alternative.

Figure 8.14. If our `flatMap` did not flatten, we would end up with a Matryoshka-doll-like collection of `Validations` inside other `Validations`. It would be extremely difficult to work with this nested type correctly.



The simplified definition for `flatMap` on a `Validation[F, S]` called `self` should look familiar after `map`:

```
def flatMap(aFunc: S => Validation[F, T]): Validation[F, T] = self match
  case Success(aValue) => aFunc(aValue)
  case Failure(fValue) => Failure(fValue)
}
```

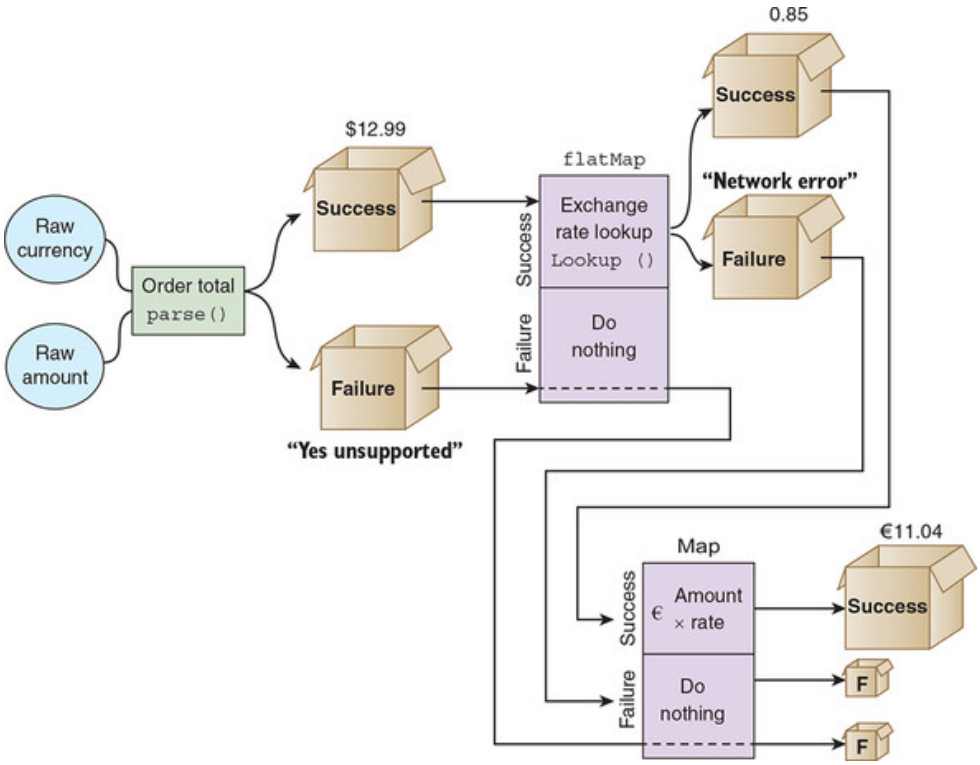
The two differences from `map` are as follows:

- The function passed to `flatMap` returns a `Validation[F, T]`, not just a `T`.

- On `Success`, we remove the original `Success` container and leave it to `aFunc` to determine whether `T` ends up in a new `Success` or `Failure`.

Putting this all together, figure 8.15 attempts to visualize how our `convert` function is working.

Figure 8.15. This visualization of our `convert` function shows how our now-familiar `Validation` boxes interact first with a `flatMap` and then with a `map`. `flatMap` allows us to chain together multiple steps that each might result in `Failure`, without nesting `Validations` inside other `Validations`. Both `flatMap` and `map` are respecting the fail-fast requirement of the pipeline, with no further processing after a Failure has occurred.



The key point to understand here is that `flatMap` and `map` support our railway-oriented processing approach across multiple work steps:

- We can compose `Validations` that are peers of each other *inside* an individual work step by using the Scream operator introduced in section 8.3.
- We can then compose multiple sequential steps that depend on the previous steps succeeding by using `flatMap` and `map` as required.
- If we are lucky enough to stay on the happy path, we can transform the value and type, safely boxed in our `Success`, as we move from work step to work step.
- Conversely, as soon as we encounter a `Failure` at the boundary between one work step and the next, we can fail fast, with our `Failure`-boxed error or errors becoming the final result of our processing flow.

This completes our look at failure handling in the unified log. If we take a moment's breath and look back over the previous three sections, we see an interesting pattern emerging, one that you could call the *compose, fail-fast, compose* sandwich (or burger, if you prefer). To explain myself:

- *Composition in the large*—We compose complex event-stream-processing workflows out of multiple stream-processing jobs, each of which outputs a happy stream and a failure stream.
- Fail-*fast as the filling or patty*—If a stream processing job consists of multiple work steps that have a dependency chain between them, we must fail fast as soon as we encounter a step that did not succeed. Scala's `flatMap` and `map` help us to do this.
- *Composition in the small*—If we have a work step inside our job that contains multiple independent tasks, we can perform all of these and then compose these into a final verdict on whether the step was a `Success` or a `Failure`. Scalaz's Scream operator, `|@|`, helps us to do this.

Summary

- Unix's concept of three standard streams, one for input and one each for good and bad output, is a powerful idea that we can apply to our own unified log processing.
- Java uses exceptions for program termination or recovery but lacks tools to elegantly address failure at the level of an individual *unit of work*. Many programmers lean on error logging to address this, which has helped to spawn the *log-industrial complex*.
- In the unified log, we can model failures as events themselves. These events should contain all of the causes of the failure, and should also contain the original event, to enable reprocessing in another stream-processing job.
- Stream processing jobs should echo the Unix approach and write successes to one stream, and failures to another stream. This allows us to compose complex processing flows out of multiple jobs.
- As a strongly typed functional language, Scala provides tools such as `flatMap`, `map`, and `for {} yield` syntactic sugar, which help us to keep our failure path integrated into our core code flow, versus exception-throwing or error-logging approaches.
- The Scalaz `Validation` is a kind of container that can represent either `Success` or `Failure` and can contain different types for each. Again, this helps us to keep our failure path co-situated with our happy path.
- The Scream operator, `|@|`, can be applied to multiple Scalaz `Validations` to compose an output `Validation`. This lets us compose multiple tasks inside a single step, into a final verdict on whether that step succeeded or failed.

- Across multiple work steps inside a single job, we want to fail fast as soon as we encounter a step that did not succeed. Scala's `flatMap` and `map` work with the `Validation` type to let us do this; Scala's `for {} yield` syntax makes our code much more readable.
- If we lean back and squint a little, we can see the overall methodology as a compose, fail-fast, compose approach, where we compose multiple jobs, fail fast between multiple steps inside the same job, and again compose between multiple tasks within the same job step.